

Arrays og Pointers - Questions

Question: Arrays 1 - Basic Array

Short Humorous Scenario: Martin ønsker at store several numbers together but doesn't know how. "Jeg skal til keep track of my lucky numbers: 2, 3, 5, 7. Hvordan do I store them all in one place?" han spørger.

Hvad de allerede ved:

- Variabel declaration
- Datatypes (`int` for whole numbers)
- Loops (`for`)
- `printf` function

Opgaven:

1. Open/create the file `arrays1.c`
2. Lav an array med `Numbers` der indeholder the values {2, 3, 5, 7}
3. Lav a loop that prints all values of the array én ad gangen
4. Compile og run the program from VS Code
5. Compile og run the program from terminal
6. **PBL:** Martin spørger: "Kan you modify the program til print the values in reverse order?"

Hint:

- If using whole numbers, `int` is the appropriate datatype
- Arrays are indexed from 0 til N-1
- `Numbers[0]` contains the value 2
- Brug a `for` loop with index `i` from 0 til array size - 1

Hvad du aldrig må gøre:

- Udskriv the array name directly (e.g., `printf("%d", Numbers)`)
- Access elements beyond the array bounds
- Glem til specify the array size

Question: Arrays 2 - Dynamic Allocation

Short Humorous Scenario: Martin har seen code that uses `malloc` og is confused. "This arrays2.c program creates arrays with `rand()`. What does this code do? And why is this dynamic allocation?" han spørger.

Hvad de allerede ved:

- Arrays

- Dynamic memory allocation: `malloc`, `calloc`
- `rand()` function
- `sizeof` operator
- Heap vs stack memory

Opgaven:

1. Compile `arrays2.c`:

```
gcc arrays2.c
```

2. Kør program from terminal:

```
./a.exe
```

3. Answer the following:

- What does the code do?
- What makes this dynamic memory allocation?
- Is this heap or stack allocation?
- What are the advantages of heap vs stack allocation?

4. **PBL:** Martin spørger: *"Kan you explain what would happen if we didn't free the allocated memory?"*

Hint:

- `dynamic_array_size = 1 + rand() % Max_array_size` creates a random size
- Dynamic allocation means the array size can be different each time
- `malloc` allocates memory from the heap
- Heap allocation allows flexible sizing
- Heap requires manual memory management (free)
- Stack is faster but fixed size

Hvad du aldrig må gøre:

- Glem til check if malloc succeeded
- Glem til free allocated memory
- Assume array size is known at compile time

Question: Pointers Basics

Short Humorous Scenario: Martin har heard about pointers but doesn't understand them. *"Jeg skal til store a memory address og use it til change a variable. Hvordan does this work?"* han spørger.

Hvad de allerede ved:

- Variabel declaration

- Memory addresses
- Dereferencing operator `*`
- Address-of operator `&`

Opgaven:

1. Lav a program `pointers1.c` med følgende variables:

- `int num`
- `int* p_num`

2. Set `p_num` til the memory address of `num`

3. Brug `p_num` til change the value of `num`

4. Udskriv both the value og address of `num` og `p_num`

5. Kør program

6. **PBL:** Martin spørger: *"Kan you create a function that takes a pointer as a parameter og modifies the original variable?"*

Hint:

- Hvordan til get a variable's address: use `&` (address-of operator)
- Hvordan til dereference a pointer: use `*` (e.g., `*p_num = 42`)
- Pointer declaration: `int* p_num` or `int *p_num`
- Udskriv address with `%p` format specifier

Hvad du aldrig må gøre:

- Glem `&` when getting address
- Glem `*` when dereferencing
- Tildel til pointer without address (wild pointer)

Question: Descriptive Statistics 1 - Sum

Short Humorous Scenario: Martin har a list of numbers og wants til add them up. *"Jeg har these numbers: 151, 123, 7, 1, -31, 0, 17, 9921, 9, 15, -1003, -3, 17. Kan you calculate the sum?"* han spørger.

Hvad de allerede ved:

- Arrays
- Loops (`for`)
- Addition operator `+`
- Variabel initialization

Opgaven:

1. Definer en array `arr` der indeholder all the numbers above

2. Definer en variable `sum` og sæt den til 0

3. Brug a loop, sum all the numbers in `arr` via the `sum` variable
4. Udskriv `sum`
5. Compile og run the program
6. **PBL:** Martin spørger: *"What is the sum of all positive numbers only?"*

Hint:

- Formula: $sum = x_0 + x_1 + \dots + x_{n-1} = \sum_{t=0}^{n-1} x_t$
- Start med `sum = 0`
- Tilføj each element: `sum += arr[i]`

Hvad du aldrig må gøre:

- Glem til initialize `sum` til 0
 - Tilføj array name directly
 - Brug wrong loop bounds
-

Question: Descriptive Statistics 2 - Mean

Short Humorous Scenario: Martin now wants til calculate the average of his numbers. *"Jeg har the sum, but what's the average? Kan you calculate that?"* han spørger.

Hvad de allerede ved:

- Arrays
- Loops
- Division operator `/`
- Sum calculation (from previous question)

Opgaven:

1. Brug `descriptive1` as a starting point
2. Beregn the mean of all numbers: `mean = sum / N`
3. Gem the mean in a variable
4. Compile og run the program from terminal og VS Code
5. **PBL:** Martin spørger: *"Kan you modify the program til calculate the mean of just the positive numbers?"*

Hint:

- Formula: $\bar{x} = \frac{1}{N} \sum_{t=0}^{n-1} x_t$
- N is the number of elements
- Brug array size: `sizeof(arr) / sizeof(arr[0])`

Hvad du aldrig må gøre:

- Divide by 0 (ensure `N > 0`)

- Brug integer division when you want decimal result
- Glem til store the result

Question: Descriptive Statistics 3 - Mean og Standard Deviation

Short Humorous Scenario: Martin ønsker at analyze patient data statistically. "Jeg har measurements for age, height, og weight. Kan you calculate mean og standard deviation for each?" han spørger.

Hvad de allerede ved:

- Arrays (including 2D arrays)
- Loops
- Arithmetic operations
- Math libraries: `math.h`
- Functions: `sqrt()`, `pow()`

Opgaven:

1. Overvej which libraries til use for x^2 og x (Hint: Is there a library with math functions?)
2. Definer en 2D `float` array `measurements` with size 3x10
3. Definer 2 1D arrays of length 3: `gennemsnit` (means) og `afvigelse` (standard deviations)
4. Definer en 1D array `measurement_names` der indeholder 3 strings: "Age", "Height", "Weight"
5. Beregn the mean of age, height, og weight from the following patient values:
 - Age: 23, 35, 67, 42, 54, 18, 29, 72, 31, 88
 - Height: 1.78, 1.62, 1.50, 1.90, 1.70, 1.55, 1.82, 1.60, 1.75, 1.85
 - Weight: 70.5, 55.3, 60.2, 85.7, 72.4, 50.1, 68.9, 64.0, 75.3, 90.2
6. Beregn standard deviation (skip this step until last)
7. Udskriv deskriptiv table in terminal (mean (std)) for age, height, og weight
8. Compile og run the program
9. **PBL:** Martin spørger: "Kan you extend this til calculate the median as well?"

Hint:

- Du har this before
- Mean formula: $\bar{x} = \frac{1}{N} \sum_{t=1}^N x_t$
- Standard deviation formula: $s = \sqrt{\frac{1}{N-1} \sum_{t=1}^N (x_t - \bar{x})^2}$
- `N_patients = 10`, `N_measurements = 3`

Hvad du aldrig må gøre:

- Beregn mean incorrectly
- Brug N instead of N-1 for standard deviation
- Glem til include `math.h`